Task 3 – From Sketch to Layout: Campus Library Search Screen (9 Marks, K4)

Run each program yourself and attach your own screenshots, each linked to the requirement or test it shows. Items marked `<fill>` need your own observations or measurements.

Task 3.1 – Layout managers and justification

Container	Layout manager	Justification with respect to resizing
Frame (outer)	`BorderLayout`	NORTH = title bar, CENTER = main area, SOUTH = buttons and status. On resize, CENTER receives all extra space while NORTH and SOUTH keep their preferred height. This gives the required "results grow" behaviour.
Title bar	label placed in NORTH	Spans the full width; height stays fixed.
Search row panel	`FlowLayout` (left)	Text field, category choice and Search button keep their preferred sizes however wide the window becomes.
Centre panel (search row + results)	`BorderLayout`	NORTH holds the fixed search row; CENTER holds the results in a scroll pane, which takes all remaining space.
Button row	`GridLayout(1,3)` inside a `FlowLayout` holder	Prev, Next and Reserve get equal sizes. The `FlowLayout` holder stops them from stretching with the window.
Bottom container	`BorderLayout`	NORTH = button holder, SOUTH = status label.
`CardLayout`	not needed here	Used for switching

Container	Layout manager	Justification with respect to resizing
		whole screens (see Task 6), not for resizing one screen.

Task 3.2 – Nested containers

```
JFrame — BorderLayout
├── NORTH   : JLabel  (title)
├── CENTER  : JPanel centre — BorderLayout
│               ├── NORTH  : JPanel searchRow — FlowLayout
│               │               ├── JTextField
│               │               ├── JComboBox (category)
│               │               └── JButton "Search"
│               └── CENTER : JScrollPane — JTextArea (results)   ← grows
└── SOUTH   : JPanel south — BorderLayout
                ├── NORTH : JPanel buttonHolder — FlowLayout
                │             └── JPanel buttons — GridLayout(1,3): Prev | Next | Reserve
                └── SOUTH : JLabel status
```

Task 3.3 – Implementation (LibrarySearch.java)

```
import javax.swing.*;
import java.awt.*;

public class LibrarySearch extends JFrame {
    public LibrarySearch() {
        super("Campus Library Search");
        setLayout(new BorderLayout());
// frame

        JLabel title = new JLabel("Campus Library",
SwingConstants.CENTER);
        title.setFont(title.getFont().deriveFont(Font.BOLD, 20f));
        add(title, BorderLayout.NORTH);
// title bar: full width, fixed height

        JPanel searchRow = new JPanel(new
FlowLayout(FlowLayout.LEFT));  // components keep preferred size
        JTextField query = new JTextField(20);
        JComboBox<String> category = new JComboBox<>(new String[]
{"All", "Title", "Author", "ISBN"});
        searchRow.add(query); searchRow.add(category);
searchRow.add(new JButton("Search"));

        JTextArea results = new JTextArea(10, 40);
        results.setEditable(false);
        JPanel centre = new JPanel(new BorderLayout());
```

```
// nested: search row top, results fill
        centre.add(searchRow, BorderLayout.NORTH);
        centre.add(new JScrollPane(results), BorderLayout.CENTER);
// CENTER takes all extra space
        add(centre, BorderLayout.CENTER);

        JPanel buttons = new JPanel(new GridLayout(1, 3, 5, 5));
// three equal buttons
        buttons.add(new JButton("Prev")); buttons.add(new
JButton("Next")); buttons.add(new JButton("Reserve"));
        JPanel buttonHolder = new JPanel(new FlowLayout());
// keeps buttons at preferred size
        buttonHolder.add(buttons);
        JPanel south = new JPanel(new BorderLayout());
        south.add(buttonHolder, BorderLayout.NORTH);
        south.add(new JLabel("Ready"), BorderLayout.SOUTH);
// status message
        add(south, BorderLayout.SOUTH);

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        pack();
    }
    public static void main(String[] a) {
SwingUtilities.invokeLater(() -> new
LibrarySearch().setVisible(true)); }
}
```

Compile and run: `javac LibrarySearch.java && java LibrarySearch`

Task 3.4 – Resizing test (take your own screenshots)

Window size	Expected behaviour (what to confirm in the screenshot)	Observed
Small (about 450 × 300)	Results area shrinks to a scroll pane; search row and buttons keep their size; the search row may wrap in `FlowLayout`.	`<fill>`
Default (after `pack()`)	All regions at their preferred sizes.	`<fill>`
Large (maximised)	Only the results area grows, filling the centre; title keeps full width; search row and buttons stay the same size; status label stays at the bottom.	`<fill>`

Alternative concept and complexity

`GridBagLayout` or absolute positioning (`setLayout(null)` with `setBounds`) instead of nested `BorderLayout`/`FlowLayout` panels:

```
setLayout(null);
title.setBounds(0, 0, 600, 30);
query.setBounds(10, 40, 300, 25);
results.setBounds(10, 75, 580, 250);
prev.setBounds(10, 335, 80, 25);                    // every position
computed by hand
```

- **Resizing:** with `null` layout nothing moves or grows when the window is resized, so the results area stays the same size, the screen looks wrong on other resolutions and fonts, and text can be clipped. `GridBagLayout` can reproduce the behaviour (results cell with `weightx = weighty = 1`, `fill = BOTH`, other cells weight 0), but needs `GridBagConstraints` for every component and is harder to read.
- **Time:** layout computation for nested managers is O(c) in the number of components c per validation, with a few small recursive passes through the nesting depth. `GridBagLayout` is also O(c) but with a larger constant (it solves weights across rows and columns). Absolute positioning does no computation at all (O(1) per resize), but you pay in broken behaviour.
- **Space:** nested design: about 6 container objects (the extra panels) plus the layout manager objects. `GridBagLayout`: one container, but one `GridBagConstraints` per component (O(c)). Null layout: one container, no layout objects. The difference is small in memory; the maintainability difference is large.